PROXY DESIGN PATTERN



Creational (5)

- [x] Factory
- [ ] Abstract Factory
- [x] Singleton
- [ ] Prototype
- [ ] Builder

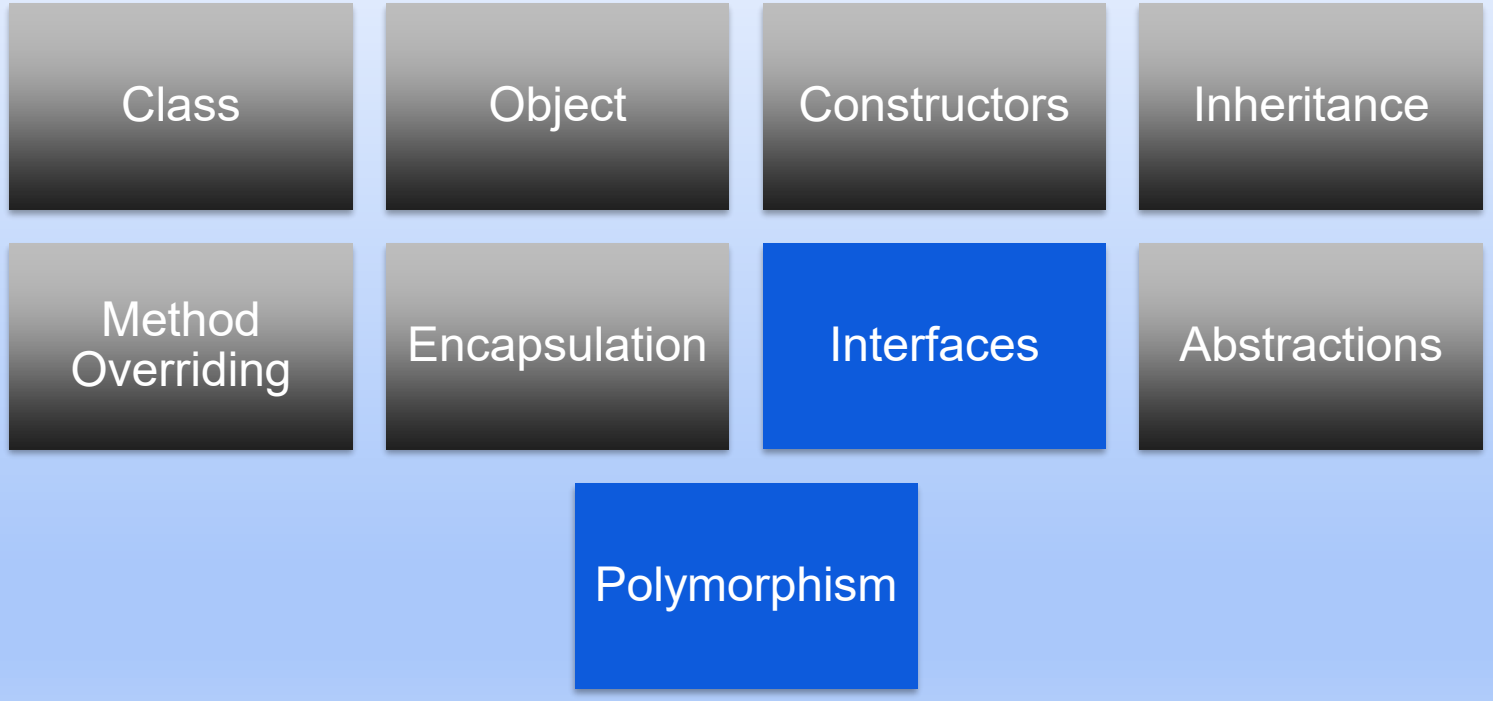
Structural (7)

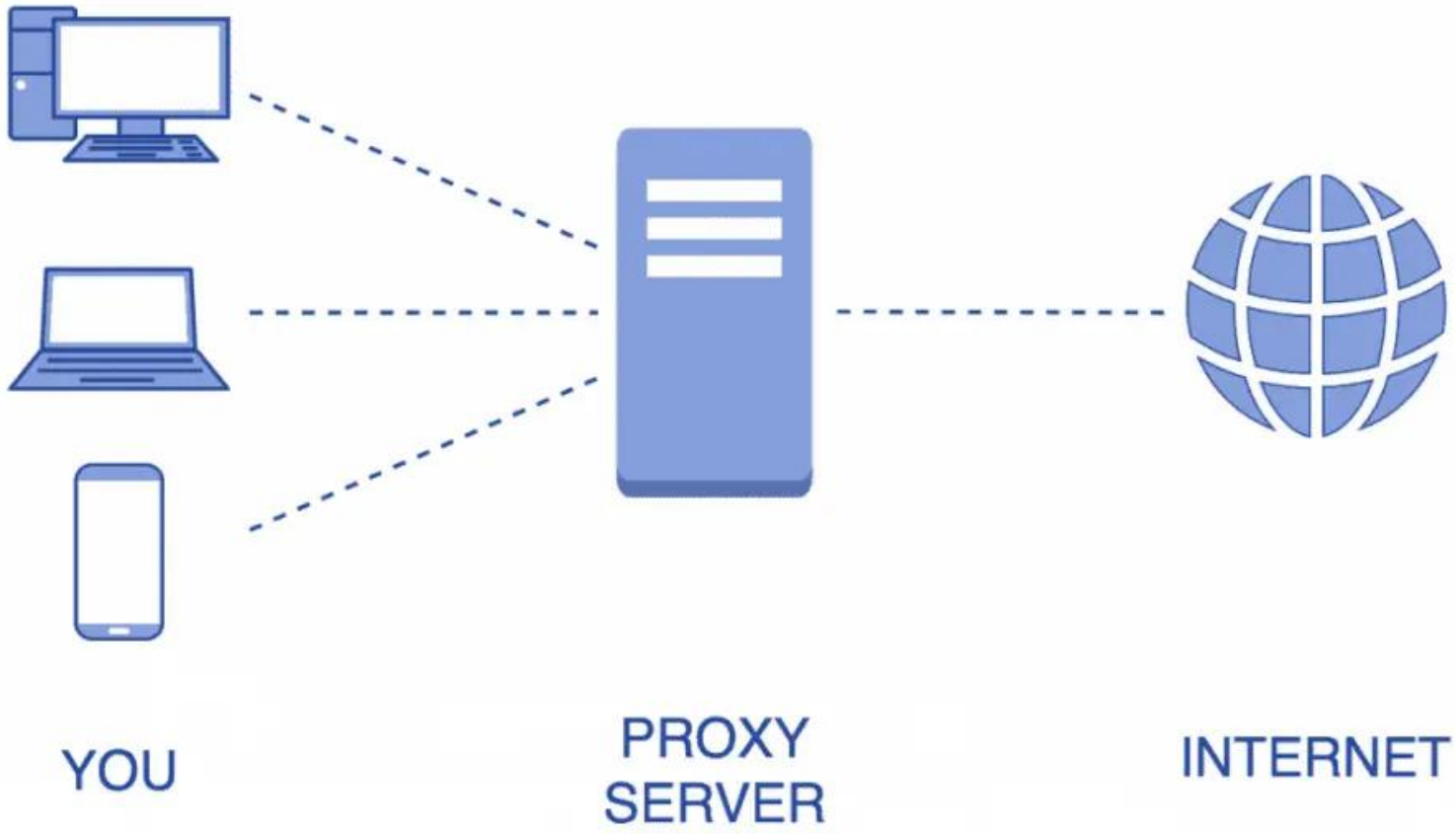
- [x] Adapter
- [ ] Bridge
- [ ] Composite
- [ ] Decorator
- [x] Facade
- [ ] Flyweight
- [ ] Proxy 

Behavioral (11)

- [ ] Chain Of Responsibility
- [ ] Command
- [ ] Interpreter
- [ ] Iterator
- [ ] Mediator
- [ ] Memento
- [ ] Observer
- [ ] State
- [ ] Strategy
- [ ] Template
- [ ] Visitor

OOP Principals




YOU
PROXY SERVER
INTERNET

What is a Proxy Server?

- A proxy server is an intermediary between a user's device and the internet.

- Key Functions:
    1. Anonymity – Hides the client's IP address
    2. Access Control – Blocks or allows content
    3. Caching – Speeds up responses & saves bandwidth
    4. Security – Filters malicious traffic
    5. Bypassing Restrictions – Access blocked/geo-restricted content

- Flow: Client → Proxy Server → Destination Server → Proxy Server → Client

Who is a Proxy?







What is Proxy Design Pattern?

This design pattern provides a substitute for another object and controls access to that object, allowing you to perform something before or after the request reaches the original object.

The proxy acts as a simplified or lightweight version of the original object.

A proxy object is still able to accomplish the same tasks.

It but may delegate requests to the original object to achieve them.

Common Scenarios



Virtual Proxy

Resource incentive operations
Lazy Loading
Image Editing
Gaming



Protection Proxy

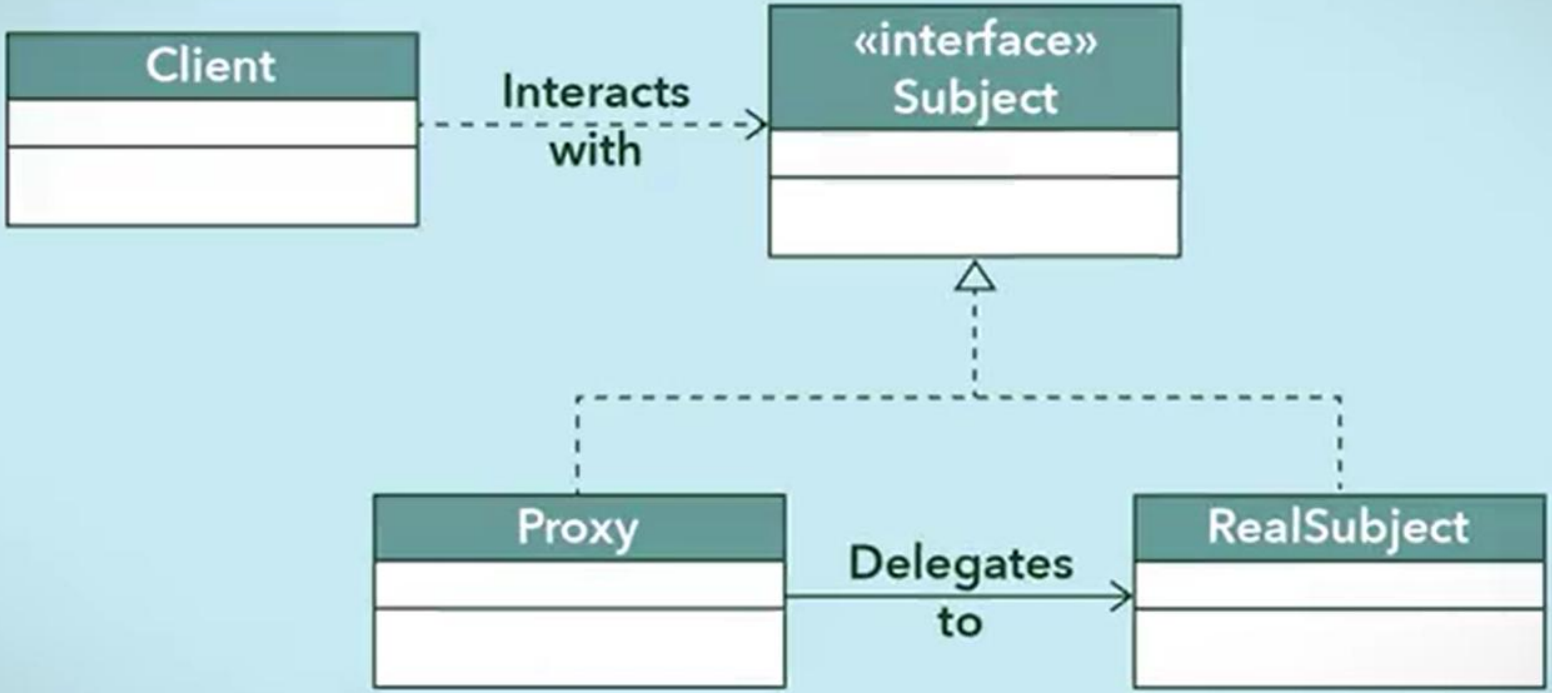
Authorization of different roles
Users can access only their roles
Data level security



Remote Proxy

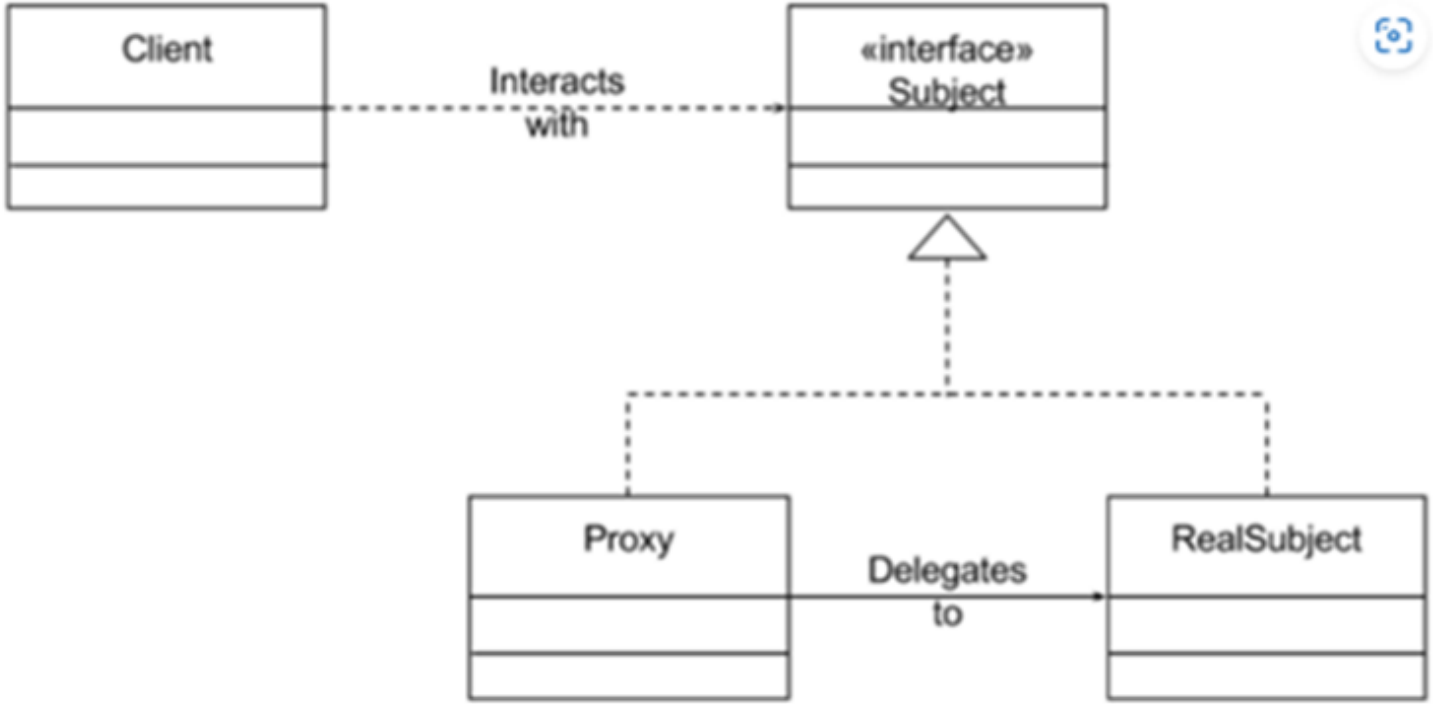
Working Remotely

Proxy class should have the same methods as the real class.


Client
Interacts with
«interface»
Subject
Proxy
Delegates to
RealSubject

Question

Which conclusions can be drawn about the proxy design pattern based on its structure as shown in the UML class diagram?



1. Proxy and RealSubject are subtypes of Subject.
2. The proxy design pattern achieves polymorphism through implementing a Subject interface.
3. The RealSubject class knows about the Proxy class, meaning that the RealSubject class has an attribute that refers to an instance of the Proxy class.
4. The proxy design pattern cannot achieve polymorphism because the Proxy and RealSubject classes are both implementing the Subject.

1. Proxy and RealSubject are subtypes of Subject.

Correct! This is clear from the way interface inheritance is shown with an open arrowhead. Since both Proxy and RealSubject must implement the methods of Subject, they subtypes.

2. The proxy design pattern achieves polymorphism through implementing a Subject interface.

Correct! Polymorphism is achieved with the interface, ensuring that both the Proxy and the RealSubject can be interacted with using the same methods, even if the internal implementation of these methods is different.

3. The RealSubject class knows about the Proxy class, meaning that the RealSubject class has an attribute that refers to an instance of the Proxy class.

4. The proxy design pattern cannot achieve polymorphism because the Proxy and RealSubject classes are both implementing the Subject.

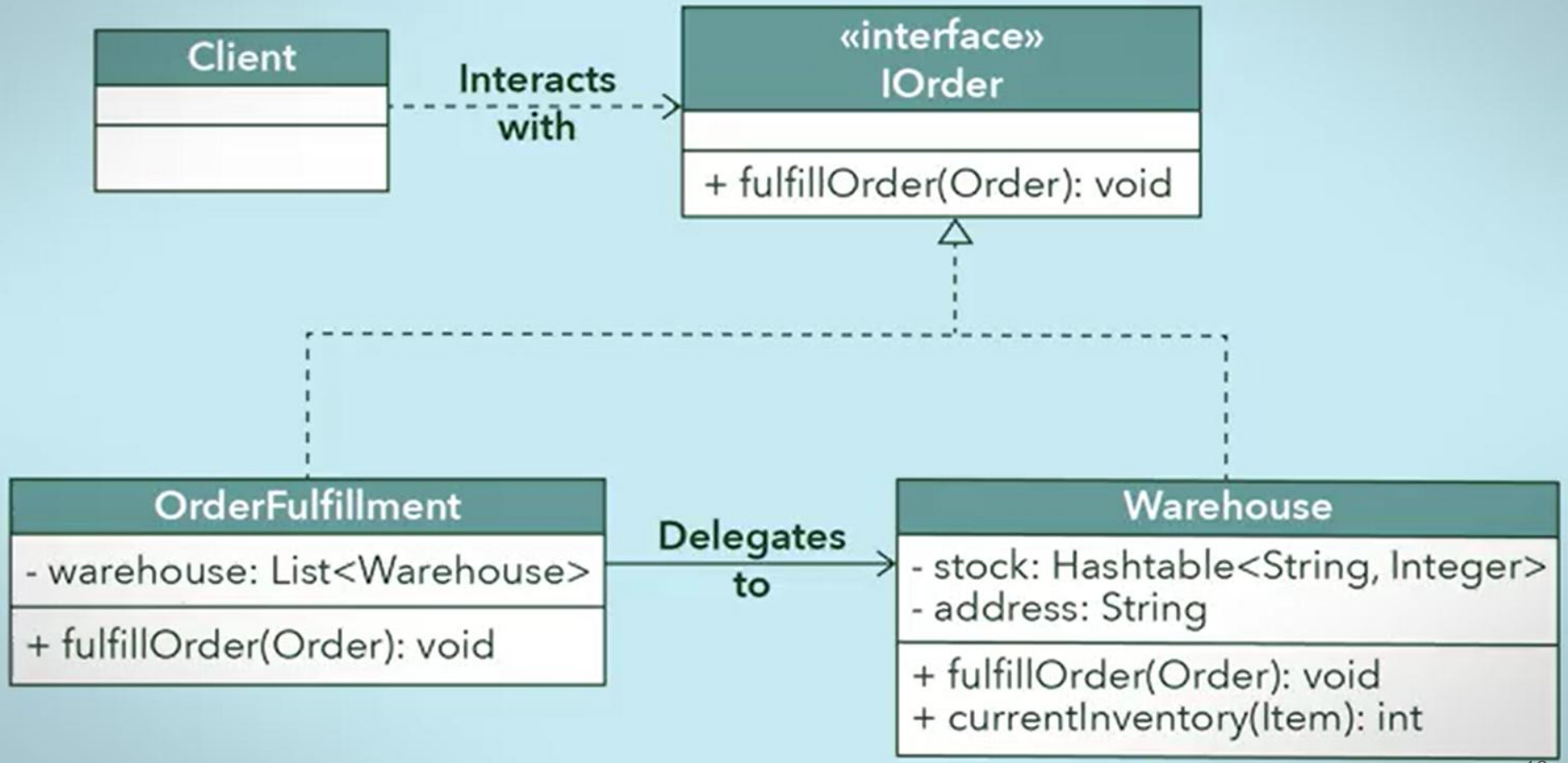
Proxy Design Pattern – Practical Application

- Scenario:
  - You run an online retail store with global distribution and warehousing.

- Problem:
  - Orders may be routed to a warehouse without stock.
  - Warehouses may receive orders they cannot fulfill.

- Solution using Proxy:
  - Both Proxy and RealSubject implement the same Subject interface.
  - Client interacts with Proxy as if it were the RealSubject.
  - Proxy determines stock availability and routes orders to the correct warehouse.
  - Ensures orders are only sent to warehouses that can fulfill them.


Client
Proxy
Warehouse


Client
Interacts with
«interface»
IOrder
+ fulfillOrder(Order): void
OrderFulfillment
- warehouse: List<Warehouse>
+ fulfillOrder(Order): void
Delegates to
Warehouse
- stock: Hashtable<String, Integer>
- address: String
+ fulfillOrder(Order): void
+ currentInventory(Item): int

Step 1: Design the subject interface

```
public interface IOrder {
     public void fulfillOrder(Order);
}
```

Step 2: Implement the real subject class

```
public class Warehouse implements IOrder {
     private Hashtable<String, Integer> stock;
     private String address;

     /* Constructors and other attributes would go here */
     …

     public void fulfillOrder(Order order) {
          for (Item item : order.itemList)
               this.stock.replace(item.sku, stock.get(item)-1);

          /* Process the order for shipment and delivery */
          …
     }

     public int currentInventory(Item item) {
          if (stock.containsKey(item.sku))
               return stock.get(item.sku).intValue();
          return 0;
     }
}
```

```
public class OrderFulfillment implements IOrder {
    private List<Warehouse> warehouses;

    /* Constructors and other attributes would go here */

    public void fulfillOrder(Order order) {
        /* For each item in a customer order, check each warehouse
        to see if it is in stock.

        If it is then create a new Order for that warehouse. Else
        check the next warehouse.

        Send the all the Orders to the warehouse(s) after you finish
        iterating over all the items in the original Order. */
        for (Item item: order.itemList) {
            for (Warehouse warehouse: warehouses) {
                …
            }
        }
        return;

    }
}
```

Step 3: Implement the proxy class

Usage of Proxy Design Pattern

- Protect the real subject class
- To have a polymorphic design so that the client class can expect the same interface for the proxy and real subject classes,
- To use a lightweight proxy in place of a resource intensive object until it is actually needed,
- To implement some form of intelligent verification of requests from client code in order to determine if, how, and to whom the requests should be forwarded to
- To present a local representation of a system that is not in the same physical or virtual space.
- Manages the lifecycle of the service object and proxy will work even if the service object isn’t ready or is not completely available.

```java
public interface Internet {
    void connectTo(String host);
}

public class RealInternet implements Internet {
    @Override
    public void connectTo(String host) {
        System.out.println("Opened connection to " + host);
    }
}
```

```java
public class ProxyInternet implements Internet {
    private static final List<String> bannedSites;
    private final Internet internet = new RealInternet();

    static {
        bannedSites = new ArrayList<>();
        bannedSites.add("banned.com");
    }

    @Override
    public void connectTo(String host) {
        if (bannedSites.contains(host)) {
            System.out.println("Access Denied!");
            return;
        }
        internet.connectTo(host);
    }
}
```

now, the user who wants to benefit from the banned-websites functionality can use the proxy ***without affecting other users***

```java
public static void main(String[] args) {
    Internet internet = new ProxyInternet();
    internet.connectTo("google.com");
    internet.connectTo("banned.com");
}
```

```java
public interface VideoDownloader {
    Video getVideo(String videoName);
}

public class RealVideoDownloader implements VideoDownloader {
    @Override
    public Video getVideo(String videoName) {
        System.out.println("Connecting to https://www.youtube.com/");
        System.out.println("Downloading Video");
        System.out.println("Retrieving Video Metadata");
        return new Video(videoName);
    }
}
```

```java
public static void main(final String[] arguments) {
    VideoDownloader videoDownloader = new RealVideoDownloader();
    videoDownloader.getVideo("geekific");
    videoDownloader.getVideo("geekific");
    videoDownloader.getVideo("likeNsub");
    videoDownloader.getVideo("likeNsub");
    videoDownloader.getVideo("geekific");
}
```



```
Connecting to https://www.youtube.com/
Downloading Video
Retrieving Video Metadata
Connecting to https://www.youtube.com/
Downloading Video
Retrieving Video Metadata
Connecting to https://www.youtube.com/
Downloading Video
Retrieving Video Metadata
Connecting to https://www.youtube.com/
Downloading Video
Retrieving Video Metadata
Connecting to https://www.youtube.com/
Downloading Video
Retrieving Video Metadata
```

```java
public interface VideoDownloader {
    Video getVideo(String videoName);
}

public class RealVideoDownloader implements VideoDownloader {
    @Override
    public Video getVideo(String videoName) {
        System.out.println("Connecting to https://www.youtube.com/");
        System.out.println("Downloading Video");
        System.out.println("Retrieving Video Metadata");
        return new Video(videoName);
    }
}
```

```java
public class ProxyVideoDownloader implements VideoDownloader {
    private final Map<String, Video> videoCache = new HashMap<>();
    private final VideoDownloader downloader = new RealVideoDownloader();

    @Override
    public Video getVideo(String videoName) {
        if (!videoCache.containsKey(videoName)) {
            videoCache.put(videoName, downloader.getVideo(videoName));
        }
        return videoCache.get(videoName);
    }
}
```

Use cases for Proxy Design Pattern

Type of Proxy	Real-World Scenarios
Virtual Proxy	- Image viewer apps (load images on demand) - Document editors (thumbnails before full load)
Remote Proxy	- Distributed systems (RMI, CORBA, gRPC) - Cloud storage (local object for remote files)
Protection Proxy	- Banking systems (control access to transactions) - Enterprise apps (restrict confidential report access)
Smart Proxy	- Caching systems (e.g., CDN edge servers) - Database connection pooling
Security Proxy	- Firewall servers (monitor/filter traffic) - API gateways (authenticate & filter requests)
Monitoring Proxy	- Payment gateways (log transactions) - Healthcare systems (track patient data access)
Smart Routing Proxy	- E-commerce fulfillment (route to correct warehouse) - CDNs (route to nearest/fastest server)

Reflective Questions

- Why is it important that both the Proxy and Real Subject implement the same interface in the Proxy design pattern?

- How does the Proxy pattern improve flexibility and security when interacting with real objects?

- In what scenarios might using a Proxy add unnecessary overhead instead of benefits?